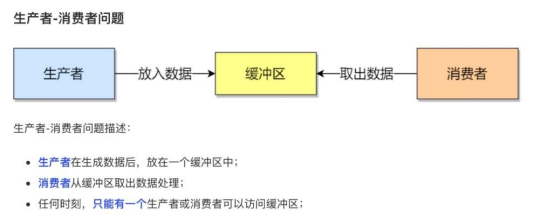
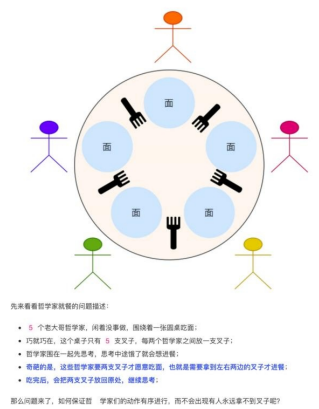
互斥，同步与锁

- **互斥与同步**
  - 互斥：保证一个线程在临界区执行的时候，其他线程应该就被阻止进入临界区
  - 同步：并发进程/线程在一些关键点上可能需要互相等待与互通消息，这种相互制约的等待与互通信息称为进程/线程同步
  - 为了能让进程和线程之间正确的解决问题，就饿可以使用两种常用的方法：锁和信号量
- **锁**
  - 自旋锁（忙等待锁）：这种锁在等待的时候就会采用while不断等待，直到空闲后就开始运行，这种锁永远不会放弃cpu
  - 无等待锁：当线程获得锁的时候，就一直在锁的等待队列，然后通过调度程序完成后续运行
- **信号量**
  - 信号量也可以控制，它可以实现线程的同步与互斥
  - 对于同步可以考虑多个信号量，对于互斥可以考虑单个信号量
- **应用**
  - 生产者-消费者问题
    - 概述



    - 问题分析
      - 任何时刻只有一个线程操作缓冲区，说明缓冲区是临界代码，需要互斥
      - 缓冲区空的时候，消费者必须等待生产者生成数据，缓冲区满时候，生产者必须等待消费者取出数据，说明生产者呃消费者需要同步
  - 哲学家就餐问题



    - 解决方案
      - 单纯的pv操作：不行，会出现死锁，就是所有人都拿左边的叉子就没有办法吃了
      - 单纯的加锁：不行，会出现浪费，只有一个人能用叉子，按道理可以两个人用的
      - 加锁加上哲学家行为控制：没问题，让偶数编号的哲学家先拿左边的叉子再拿右边的叉子，奇数编号的哲学家先拿右边的叉子再拿左边的叉子，然后再加锁就可以正常进餐了
  - 读者--写者问题
    - 概述

读者只会读取数据，不会修改数据，而写者即可以读也可以修改数据。

读者-写者的问题描述：

- 「读-读」允许：同一时刻，允许多个读者同时读
- 「读-写」互斥：没有写者时读者才能读，没有读者时写者才能写
- 「写-写」互斥：没有其他写者时，写者才能写

    - 解决方案
      - 读者优先策略：优先让读者写完所有数据，如果有读者来了写者就阻塞等待读者做完再写入
      - 写者优先策略：与读者相反
      - 公平策略：类似于deque，写者读者挨个访问，但是允许多个连续的 读者同时访问